



# A Machine Learning Approach for Predicting Student Progress in Online Programming Education

Vincent Zhang<sup>1</sup> · Bryn Jeffries<sup>1</sup> · Irena Koprinska<sup>1</sup>

Received: 3 October 2024 / Accepted: 12 August 2025 / Published online: 25 September 2025

© The Author(s) 2025

## Abstract

Student data from online programming courses can provide valuable insights into how students approach problems and acquire critical computational skills, as well as about the challenges they face in the process. However, extracting these insights from data can be challenging, especially when dealing with large, complex and multi-dimensional datasets. In this paper, we propose a machine learning approach to predicting student progress at a module level in large-scale online programming courses. Our approach defines suitable content interaction features from the log data that measure student engagement with course material, and then creates a decision tree classifier to predict performance on the last problem in the module. Using data from four large-scale programming courses for upper primary and high school students, we demonstrate that this approach can produce accurate predictions of student progress and dropouts. Additionally, our approach provides interpretable tree-based visualisations that identify key course materials and programming tasks for successful course completion, and also highlights differences in student behaviour between courses. The predictive models were found to be informative by educators for improving the design of these online courses. The intrinsically explainable decision trees provided competitive accuracy compared to more advanced black-box models. By providing insights into the pedagogical value of course content, our approach facilitates a data-driven approach to increasing learning outcomes.

**Keywords** Student progress · Computer programming · Machine learning · Decision trees

---

✉ Vincent Zhang  
vincent.zhang@sydney.edu.au

Bryn Jeffries  
bryn.jeffries@sydney.edu.au

Irena Koprinska  
irena.koprinska@sydney.edu.au

<sup>1</sup> School of Computer Science, The University of Sydney, Sydney, NSW 2006, Australia

## Introduction

Programming and computational skills are highly in-demand skills in the new digital economy and technology-advanced society. Globally, many countries around the world have made increasing the computational skills and literacy of their K-12 students a priority (Vegas et al., 2021). However, mastering these skills is difficult, particularly for K-12 students with no prior programming experience.

As schools become more digital, vast amounts of rich student data are being generated, and tools based on artificial intelligence and learning analytics are becoming more common. In particular, many schools are using online coding education platforms provided by organisations such as Code.org in the USA, and Raspberry Pi Foundation in the UK, to teach their students computational skills. These platforms provide interactive coding environments where students can solve programming tasks and write, test, debug and automark their code. They also collect large amounts of useful student data, and there is an opportunity to apply educational data mining techniques to analyse this data. Doing so can help educators better monitor student progress, understand learning patterns and design more effective strategies and interventions to improve student learning outcomes and prevent dropout.

Student programming data is often large, complex, noisy and multi-dimensional. This is further complicated by group and individual differences in how students learn and interact with coding platforms, and the degree to which this affects their performance. These factors can make it challenging for educators to process the data and extract valuable insights in an efficient and interpretable way.

Existing research mainly focuses on university programming courses, but there is a growing shift towards teaching coding at K-12. Students' career inclinations develop earlier, in school. Providing better support for K-12 students undertaking programming courses can improve their confidence and encourage more students to pursue computing courses and computer science degrees in the future.

This paper proposes a decision tree approach to predicting student progress and identifying important course material for students to focus on at a *course module* level. Specifically, it predicts the student performance on the last problem in the module, based on the student interactions with the previous content in the module. We chose decision trees as they are intrinsically interpretable (glass-box) models and are also computationally efficient to build. Our approach defines and extracts suitable content interaction features from the student log data, representing the degree of interaction that each student has with the material in the course module. The most important subset of these features is then selected to create the decision tree.

For each module, our approach produces an interpretable decision tree, as well as a ranking of the most important course materials. This provides educators with a module-level understanding of student progress and which course materials students should focus on to improve their progress. It also identifies key places where students might be struggling and give up, and programming tasks that are predictive of failure and success and should be completed. The evaluation is conducted using student log data from four large-scale online programming courses for upper primary and high school students, representing a wide sample of students of varying abilities and backgrounds.

Overall, we aim to answer the question: *How can we make predicting K-12 student progress in programming courses more accurate and interpretable using machine learning techniques?* This is guided by four research questions:

1. Are course content interaction features a useful predictor of student outcomes at a module level?
2. How can we represent student progress in a useful format for educators?
3. What are the important course materials for successful module completion?
4. To what extent is this approach and the results useful for educators?

This paper is set out as follows: “[Background and Related Work](#)” section examines the background and discusses how this work builds on related work. “[Data](#)” section describes the dataset. “[Exploratory Data Analysis](#)” section highlights important observations of student interaction patterns that motivate this work and the method. “[Predicting Student Progress](#)” section presents the method for extracting the relevant course content interaction features from the data, selecting the important features, and constructing the decision trees. “[Results](#)” section presents the key findings, and “[Discussion](#)” section discusses the results, demonstrates their implications and how they answer the research questions.

## Background and Related Work

Understanding student behaviour and student progress in online programming courses is essential for improving learning outcomes. This is especially critical for novice programmers, in order to detect students at risk of failing or dropping out and take timely remedial actions. Understanding student interactions with course activities can also help educators identify challenging concepts and misconceptions, and design new content to better target and explain these areas; present the learning activities in a different sequence or improve the ones that were found to be of little value to optimise student learning.

There is a growing body of data-driven approaches for analysing data of students using online learning environments, understanding student behaviours and providing useful insights for teachers to improve student learning and teaching resources. Below we review the main lines of research related to our work.

**Characteristics of Novice Programmers** Novice programmers tend to have limited, surface-level knowledge of programming and computer science concepts. This includes fragile knowledge (Winslow, 1996) – knowledge that the student has gained but fails to use when necessary, which can be missing (forgotten), inert (learned but not used), or misplaced (learned but used inappropriately). This lack of knowledge can prevent novice programmers from selecting the right approaches to solve problems effectively. In addition, novice programmers interact with coding problems in different ways compared to more advanced students, often approaching problems line-by-line and focusing on minor syntactic and semantic features. Previous work has largely focused on identifying knowledge gaps that explain novice programming performance, such as difficulty with recursion (Dale, 2006) or confusing equality with

assignment (Bonar & Soloway, 1985). In this work, we identify general behavioural characteristics of novice programmers and compare them to more advanced students.

**Clustering-based Approaches for Understanding Student Behaviour** Clustering approaches aim to identify typical student behaviours associated with positive and negative outcomes, and how these behaviours develop over time. For example, Perera et al. (2009) demonstrated that clustering techniques can help identify key leadership and teamwork traits among students in university-level software development projects. McBroom et al. (2016) showed that clustering methods can help identify different strategies used by students to complete programming tasks, as well as behavioural patterns throughout the semester that are associated with final exam outcomes. Similarly, temporal clustering approaches were used in McBroom et al. (2020) to identify student behavioural trends over time in high school programming courses.

**Early Prediction of Exam Grades** Research on early prediction of student grades is also highly relevant, because it enables timely and targeted interventions to support at-risk students. Early prediction of final student grades (Pass or Fail) based on student participation in a discussion forum was proposed in Romero et al. (2013). Three types of features were extracted - based on (i) message statistics, such as the number of messages read/posted, (ii) relevance of the message as judged by teachers and (ii) social network measures about the student such as centrality. Early prediction of students at risk of failing the final exam in a first year programming course based on assessment marks, auto-grading system data and discussion board participation, was investigated in Koprinska et al. (2015a). The results showed that it was possible to accurately predict the exam mark in the middle of the semester based on the available assessment marks, and that adding information from the discussion board and auto-grading system improved the predictive accuracy. In Koprinska et al. (2015b), data from the same sources was used to define the behavioural characteristics of high, average and low performing students.

Conijn et al. (2017) also predicted the final exam grade at different stages during the semester but they used LMS interaction data (e.g. views, clicks, time) and assessment marks. They found that the assessment marks were more predictive than the LMS data, on a large scale evaluation of multiple courses. Tomasevic et al. (2020) used the Open University Learning Analytics Dataset (OULAD) (Kuzilek et al., 2017) to predict the final exam mark (Fail or Pass) based on the three data sources used in Kuzilek et al. (2015) (previous assessment marks, clicks on resources and demographic data), at different time points: after each of the 6 assessments during the semester and just before the final exam. They found the neural networks performed the best obtaining an F1 score between 78% (after assignment 1) and 95% (after assignment 6) and 97% just before the exam. The demographic data was least predictive but its addition improved the performance.

**Early Prediction of Course Dropout** Lykourantzou et al. (2009) predicted dropout in online computer networks and web design courses, using rich information from student logs. The extracted features included demographic data, prior academic performance, assessment marks and submission times during the course and activity on the platform (e.g. views, posts). They achieved moderate-to-high accuracy very early in the

semester, although using non-interpretable methods such as neural networks and support vector machines. Pereira et al. (2019) predicted student dropout in introductory programming courses at a Brazilian university using data from the first few weeks based on data extracted from an automated grading system. They achieved accuracy of 80% using decision trees and suggested presenting the learned rules as pop-up message while using the system warning them of behaviours that are likely to lead to drop-out. Our approach provides an ample prediction window that provides teachers with an opportunity to intervene in real-time.

**Predicting Next Assignment or Aggregated Assignment Performance** Kuzilek et al. (2015) predicted the students at risk of failing or not submitting the next assessment based on (i) their results on the previous assessments, (ii) LMS interaction data (number of clicks on different resources) and (iii) demographic data. An ensemble combining the predictions of naive Bayes, nearest neighbor and decision tree algorithms was employed. The evaluation was conducted on two courses from the OULAD dataset achieving F1 score of 27–57%. Although the combined ensemble is not interpretable, the individual decision tree and nearest neighbor classifiers were used to provide explanations and show similar students in the dashboard.

Riestra-González et al. (2021) predicted student performance (Poor or Excellent) from log data at different stages during the semester using online behaviour data (views, clicks, time, number of attempts) and previous assessment marks. A large scale evaluation was conducted on data from all courses at the University of Oviedo during an academic year. Neural Networks obtained the best performance - F1 score of 86% at the 10% course length and 94% at the 50%. However, the target variable was defined as an aggregation of all assignment marks during the semester and the assignment marks before the prediction time were used as independent variables, which might have biased the results.

**Early Prediction of Degree Dropout** Predicting dropout at a degree level is also relevant and has been an active area of research. Kemper et al. (2020) predicted dropout at a degree level (graduation vs dropout) based on demographic data and previous semester academic results, achieving good results even after one semester was completed only. The most predictive features were the number of previous passed and failed exams and the average student grade. Berens et al. (2019) developed an early detection system for German universities. Using an AdaBoost ensemble which combined the predictions of logistic regression, neural network and random forest, they achieved 67% accuracy based on demographic data only, available at enrolment, and 83% after adding performance data from the first four semesters. A comparison of different classification algorithms and feature sets for predicting dropout at a degree level based on academic performance was also investigated in Wagner et al. (2023). The results showed high accuracy, approaching 90%, in many cases, with logistic regression performing the best. A fairness evaluation in terms of study programs and gender was also conducted. Building upon related work, our approach not only predicts dropout but also identifies key points in courses where teachers can intervene to improve student performance and prevent dropout.

**Knowledge Tracing** Another relevant area of prior work is knowledge tracing. Given a sequence of questions and the student answers to these questions, it aims to estimate the student knowledge and predict the student answer to the next question. Temporal models based on Bayesian knowledge tracing (Corbett & Anderson, 1995) are used to model the student knowledge state and hidden Markov models and Bayesian networks are used to update it, with numerous improvements explored, e.g. Kaser et al. (2017); Pardos and Heffernan (2010); Yudelson et al. (2013). Recently, deep knowledge tracing using recurrent neural networks (Piech et al., 2015) has been proposed as an alternative and shown to outperform the traditional methods. A recent survey on knowledge tracing methods is presented in Abdelrahman et al. (2023). In this work, instead of predicting student answers to specific questions, we focus on how students approach solving coding problems.

**Interpretable Methods** Developing interpretable models is critical for AI applications in education, as they can help improve stakeholder trust, support human-in-the-loop decision-making and provide actionable and understandable feedback. A recent paper highlighting this need is Dsilva et al. (2023). It applied Explainable Boosted Machines (EBM) to predict academic risk (fail or drop-out) using the OULAD dataset, and features based on demographic, pre-course and click-stream data from the learning environment. EBMs are ensembles of decisions trees. Although by combining multiple decision trees the intrinsic interpretability of a single tree is lost, EMBs are able to produce feature rankings and identify the most important features for failure and dropout prediction. These features can be tracked as the semester progresses, thereby improving understanding of student behaviour. An interpretable method for tracking student progress called progress networks (McBroom et al., 2021) has been recently proposed. Progress networks provide a visualisation which summarises the progression of a student population through a programming task. In this work, we build on previous work by creating inherently interpretable, decision tree models for predicting student progress.

## Contribution

In summary, most of previous work focuses on early prediction of exam grades and course/degree dropout, using variety of machine learning approaches. Our task is different: we consider predicting student progress at a course module level, and focus on creating interpretable classifiers. The main contributions of our work are:

1. Exploring the task of predicting student progress at a module level. Specifically, we predict the student performance on the last problem in the module, based on the student interactions with the previous content in the module.
2. Defining and extracting suitable course content interaction features, representing the degree of interaction of the student with the material.
3. Developing an approach which includes feature subset selection and decision tree classifier, and demonstrating that it produces comparable performance to more advanced black-box classifiers, while providing explainability for teachers.

4. Evaluating the performance of our approach on four large scale K-12 programming courses with thousands of students. Most of previous work focuses on university level courses and smaller cohorts.
5. Demonstrating the utility of our approach to identify key points in courses where teachers can intervene to improve student performance and prevent dropout, and key course materials and programming tasks that are predictive of failure and success.

This paper is an extension of our short conference paper (Zhang et al., 2023). Here we describe the method in more detail, provide more information about the educational context and data characteristics, present new results on interaction patterns, new results using additional performance measures (recall, precision and F1 score), comparison with other methods (Logistic Regression, Random Forest and Support Vector Machine), comparison between courses and evaluation by educators, and extend the discussion of the results by providing new observations and insights.

## Data

We use a large dataset from four online Python programming courses, which run over a five-week period in 2018. The data was provided with all student details stripped, and each student's identifier substituted with a randomly generated value to prevent re-identification.

The four courses were aligned to different age groups and abilities:

1. **Novice Blockly:** Designed for young students (10 – 14 years old) with no prior programming experience, and conducted in the Blockly visual environment.
2. **Beginners Blockly:** Designed for slightly older students (12 – 16 years old) with no prior programming experience, and conducted in the Blockly visual environment.
3. **Beginners Python:** Same problems and target group as Beginners Blockly, but conducted in Python.
4. **Intermediate Python:** Designed for high school students (14 – 18 years old) with prior programming experience. Conducted in Python.

Each course consists of 10 modules, with two modules released every week. The 10 modules for each course are denoted as w1p1, w1p2, ..., w5p2, e.g. w4p2 is the second module in the fourth week. In total, there are 40 modules across all four courses.

The course content for each module includes a number of *content slides* and *problem slides* that are focused around specific concepts and skills (Fig. 1).

The content slides provide useful information such as hints, instructions and relevant programming knowledge for students to complete the problem slides in the module. They also typically contain interactive example code that students can step through and interact with. For instance, Fig. 2 shows a content slide from the Beginners Python course introducing students to content on control flows, while Fig. 3a shows the subsequent slide which allows students to tinker with an existing control flow structure, run code, modify variables and interact with the program output. The Blockly equivalent of the second slide can be seen in Fig. 3b. Blockly is a visual environment which uses graphical blocks to represent programming concepts. The code

**Fig. 1** A module with content slides (circle symbols) and problem slides (diamond symbols)

## 2.1 –Week 2 Part 1

No points available

2.1

### Making decisions

- Why do we need decisions?
- What if it is raining?
- Controlling a block of code
- Assignment vs. comparison

### Shotgun!

### Decisions with two options

- True or False?
- Decisions with two options
- If it isn't raining...
- I Am Katniss

### Decisions about numbers

- How do we compare things?
- Experimenting with comparison
- Making decisions with numbers
- Basketball Bunny!

### Making complex decisions

- Decisions within decisions
- Decisions with multiple options
- Interplanetary visitor
- Over and Out



## Why do we need decisions?

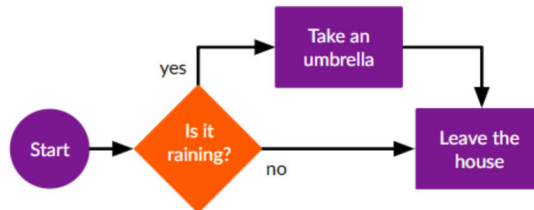


Narrate

So far our programs have been a simple sequence of steps. The interpreter executes the statements from top to bottom, and so the program *runs the same way every time*.

In the real world, we **decide to take different steps** based on our situation. For example, if it's raining, we do an *extra step* of taking an umbrella before leaving the house.

This *flowchart* describes this process (or *algorithm*):



The diamond requires a **yes** or **no** decision. The answer determines which line we follow. If the answer is **yes**, we do the extra step of taking an umbrella. If the answer is **no**, we skip it.

We can write this in Python using an **if** statement.

**Fig. 2** Example of a content-only content slide for Beginners Python course

<

What if it is raining?

>

Let's write our flowchart as a Python program:

```
raining = input('Is it raining (yes/no)? ')
if raining == 'yes':
    print('Take an umbrella.')
    print('Leave the house.')
```

Try it! What happens when you say yes, no, or any other answer?

Notice that the first `print` is *indented* (by two spaces). This is the *body* of the `if` statement. The body must be indented.

If the value stored in `raining` is equal to `'yes'` (because the user entered `yes`), then the body is run. Otherwise, it is skipped.

The second `print` always runs, because it is not indented, and isn't controlled by the `if` statement.

 **An `if` statement is a control structure**

The `if` statement controls how the program runs by deciding if the body is run or not.

(a) Interactive content slides for Beginners (in Python)

<

What if it is raining?

>

 Narrate

Now we can write a program for our flowchart!

We can check if the contents of `raining` is equal to `'yes'`:

```
set (raining) to ask: "Is it raining (yes/no)?"
if (raining) == "yes"
do
  print "Take an umbrella"
  print "Leave the house."
```

Is it raining (yes/no)? yes  
Take an umbrella.  
Leave the house.

Try it! What happens when you say yes, no, or any other answer?

If the user enters yes, the `raining` variable will be equal to `'yes'` and so `Take an umbrella.` is printed. If the user enters no, or anything else, it isn't printed.

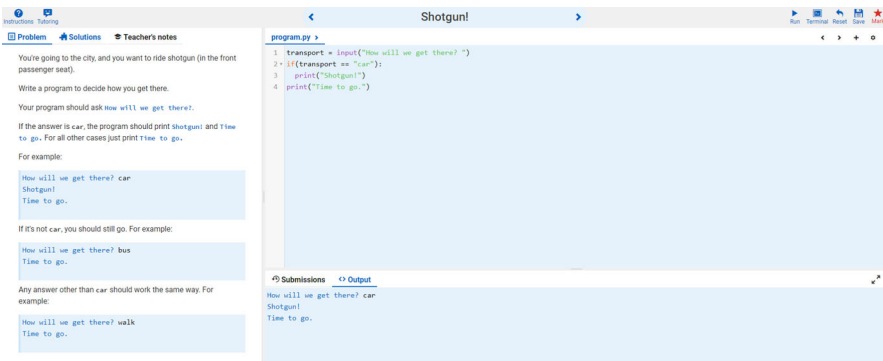
The second `print` always runs, because it isn't inside the `if` block.

 **An `if` block is a control structure**

The `if` block controls the program by deciding which blocks run.

(b) Equivalent interactive content slides for Beginners Blockly (in Blockly)

Fig. 3 Examples of interactive content slides for Beginners Python and Beginners Blockly courses



**Fig. 4** Example of a problem slide for Beginners Python (in Python)

is generated by dragging and dropping these blocks. It allows novice programmers to focus on the programming logic, while ignoring the syntax.

The problem slides occur less frequently than content slides, and are typically found at the middle and end of each module. They are designed to test the students' understanding of programming concepts and skills that they would have gained from completing the module.

Students receive points towards their final score for the course by completing the problem slides for each module. This requires students to submit code that passes all the tests. Each problem slide is worth 10 points, with the available points dropping by one for every 5 incorrect submissions to that problem slide, but dropping no lower than 5 points. This scoring system is designed to encourage students to avoid re-submitting excessively without making use of the feedback returned from the testing.

For instance, Fig. 4 shows a problem slide in Python from the same module as the slides noted above, that aims to reinforce the students' understanding of control flow. The left side of the problem slide contains a description of the problem and examples of expected output. The right side of the problem slide contains space for students to construct their solution. Students can run, save and submit (mark) their code using the respective buttons on the top right corner of the problem slide.

In total, there are approximately 35,000 students enrolled across all four courses, and within each course there are varying numbers of enrolled students. Within each module in any given course, there are varying numbers of enrolled students who are also active and participate at various points in the module. For instance, as seen in Table 1,

**Table 1** Course enrolments and number of active participants (averaged across all modules of course)

| Course              | Students enrolled | Average number and share of active students |
|---------------------|-------------------|---------------------------------------------|
| Novice Blockly      | 9,237             | 1,952 (20%)                                 |
| Beginners Blockly   | 5,980             | 1,173 (20%)                                 |
| Beginners Python    | 12,146            | 5,509 (45%)                                 |
| Intermediate Python | 6,770             | 3,330 (50%)                                 |

over 9,000 students were enrolled in the Novice Blockly course, but each module on average had 20% of students participate. Students participating in the courses were almost entirely from Australia, but a small number were from other countries such as New Zealand and United Kingdom.

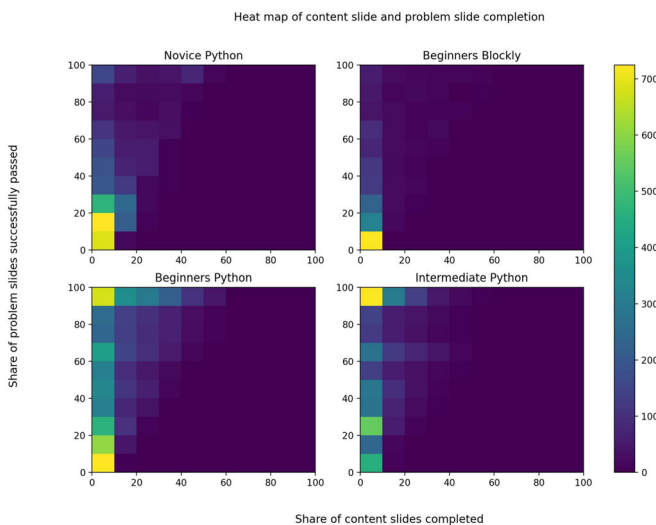
In summary, the dataset captures a wide array of students spanning different ages, grade levels and programming abilities. The dataset also captures a range of programming concepts and skills, as it includes all 10 modules, which cover different areas of programming.

## Exploratory Data Analysis

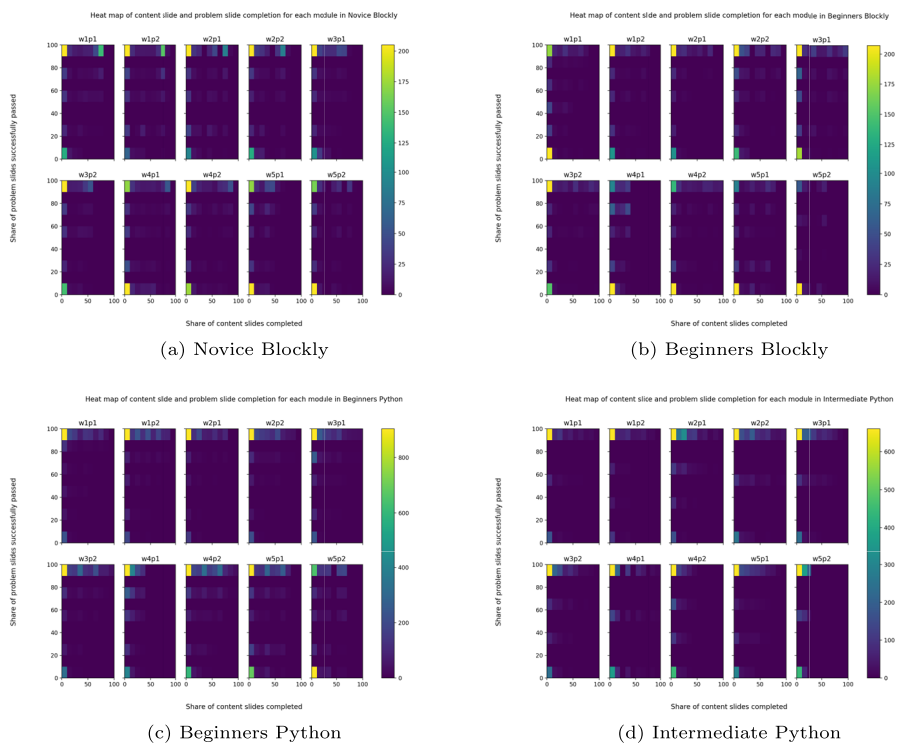
In this section, we examine the key patterns emerging from student interactions with course slides. We conduct this analysis by plotting heat maps showing problem slide completion against content slide completion, first at the course level (Fig. 5) and then at the module level (Fig. 6). Each heat map shows regions of varying student densities (i.e., the number of students that fall within the region). The brighter the region, the higher its density. We identify three groups of students from these heatmaps as summarized in Table 2.

### Course-level Interaction Patterns

There are several interaction patterns that emerge at the course-level. First, we identify a large group of disengaged, at-risk students across all courses. We define this group as students who have less than 10% of content slides, and completed between 0–30% of problem slides. These students are represented in the bottom-left corners of each heatmap, which are highlighted bright yellow/green (Fig. 5). Early identification



**Fig. 5** Course-level heat maps of problem slide and content slide completion



**Fig. 6** Module-level heat maps of problem slide and content slide completion

of these at-risk students will help provide more opportunities for educators to intervene and provide additional support. As part of the overall aim of predicting student progress, this work will focus on identifying these at-risk students.

Second, we identify an additional group of disengaged, successful students who are able to do well in the course despite not engaging with most content slides. We define this group as students who successfully pass more than 90% of problem slides, while completing less than 10% of content slides. These students are represented in the top-left corner of the heatmaps for the more advanced courses (i.e. Beginners Python and Intermediate Python). The presence of these disengaged but successful students is expected since both the Beginners Python and Intermediate Python courses are more suited to more advanced programmers. They target older students and use Python, compared to the other courses which use Blockly, a visual block-based programming environment. Block-based environments are often preferred by novice programmers

**Table 2** Slide completion ranges by student group

| Student group          | Problem slide completion range | Content slide completion range |
|------------------------|--------------------------------|--------------------------------|
| Disengaged, at-risk    | 0 – 30%                        | 0 – 10%                        |
| Disengaged, successful | 90 – 100%                      | 0 – 10%                        |
| Engaged, successful    | 90 – 100%                      | 40 – 60%                       |

due to greater ease of readability, the useful shape and visual layout of blocks, and the increased support for trial and error (Weintrop & Wilensky, 2017).

Therefore, the Beginners Python and Intermediate Python courses are more likely to have a higher representation of students with prior programming experience, who may have not found much benefit in the content slides and were able to pass problem slides by relying on their own programming skills. In particular, we compared the heatmaps for Beginners Python against Beginners Blockly, which are the same and share the same slides, except Beginners Python uses Python instead of Blockly. We observe that there is a higher density of disengaged but successful students in the top-left of the Beginners Python heat map. This result provides useful context for modelling student progress — a lack of content slide completion can be linked to both positive outcomes (high problem slide completion) and negative outcomes (low problem slide completion).

Third, content slide completion is low across all courses, as seen in the low density in the right half of each heat map in Fig. 5. Most students completed 0–20% of content slides, with some students completing up to 40% of content slides, as seen in the heat map for the Beginners Python course. This suggests that students are likely being selective in their approach to completing slides. Therefore, in the context of modelling student progress, it will be important to identify the few important content slides that students focus on. In this work, we aim to identify these key slides through feature selection (“[Feature Selection and Ranking](#)” section).

## Module-level Interaction Patterns

In addition, we analyse the heat maps of problem slide and content slide completion for each module in all four courses. There are a number of key interaction patterns that can be observed. We identify a third group of students – high-performing and engaged students – who complete both problem slides and content slides. We define this group as students who have completed more than 90% of problem slides and between 40% and 60% of content slides. These students were not observed in the course-level heat maps. For instance, the heat maps for a number of modules in the Novice Blockly courses (modules w1p1, w1p2 and w2p1, w2p2) and Beginners Python courses (modules w4p2 and w5p1) contain an additional group of students near the top-right corner, representing students who completed most content slides and performed well on problem slides (Fig. 6a). This is an important result for modelling student progress because it suggests that active engagement with certain content slides can be linked to successful outcomes on problem slides. Therefore, this result motivates our selection of the completion of key slides as one predictor of student programming outcomes.

Second, there are more disengaged, high-performers in the more advanced courses — Beginners Python and Intermediate Python. Across all modules for these courses, there is a higher representation of disengaged, high-performers than disengaged, at-risk students (Fig. 6c and d). In comparison, the module-level heat maps for the Novice Blockly and Beginners Blockly courses show a more even representation between the disengaged, high-performers and at-risk students. Understanding the differences

between students in these groups is useful context for comparing prediction performance across courses.

### Implications for Modelling Student Progress

In this section, we observed a number of important student interaction patterns. At the course-level, we observed two distinct groups of students. The first were disengaged, at-risk students who did not engage with slide content and performed poorly on problem slides. Predicting student programming progress for these disengaged, at-risk students will be critical for preventing dropout. Therefore, this work aims, in part, to predict progress for these students and report dropout (“No submission”) accuracy for these students in the results (“[Results](#)” section), alongside other metrics.

The second group of students identified at the course-level were disengaged but high-performing students who did not engage with slide content but performed well. This result provides useful context around a lack of content slide engagement, by showing that it can be linked to both negative and positive student programming outcomes. Therefore, this result shows that there are high-performing students who may require less direct assistance or even further extension opportunities, even though they are not as engaged as others with the content. Further, we found that students were likely being selective when engaging with content slides. Therefore, this work will identify the key content slides that students choose to focus on through feature selection (“[Feature Selection and Ranking](#)” section).

At the module-level, we found a third group of engaged and high-performing students who completed most content slides and passed most problem slides. This result motivates our choice for selecting completion of important content slides as one predictor of student programming progress, alongside problem slides preceding the final problem slide. We also observed differences in the types of students across courses, with the more advanced courses such as Beginners Python and Intermediate Python containing a higher share of disengaged, high-performers than disengaged, at-risk students compared to the other, beginner block-based courses. This is useful context to consider when evaluating prediction performance across courses.

### Predicting Student Progress

Our approach predicts student performance on the final problem slide in each module, using the interactions that students have with the preceding slides in the same module. Predicting end-of-module outcomes is more valuable than predicting final outcomes for the course, as many students do not complete the entire course. The three stages to our approach are outlined below.

### Extracting Student Interaction Features

For each module, we create an interaction vector for each student that represents the student’s interactions with the course content (slides). Each entry of the vector

represents a content slide or problem slide event, which is recorded when a student interacts with the slide. There are 9 possible interaction events that are recorded in the online environment:

1. Visited a content slide
2. Narrated a content slide
3. Stepped through code examples for a content slide
4. Ran own code for content slide
5. Completed a content slide
6. Visited a problem slide
7. Ran a problem
8. Failed a problem
9. Passed a problem

For content slides, possible events include slide visits, slide narrations and the running of any code examples contained therein (Events 1–5). Some content slides contain a sequence of interactive steps that the student could complete, such as running code or making specific changes to it. Each completed step yields a visual reward of the step number turning green. Completing all steps for a slide generates a celebration event which is recorded (Event 4). For problem slides, recorded events include visits, running and the submission of code for testing (Events 6–9). The outcome of any test (“Pass” if all test cases passed, otherwise “Fail”) was also recorded.

We consider the subset of these events that indicate whether a student has engaged with a content or problem slide. Specifically, the content slide events we consider are “Completed”/“Not completed”, representing whether the student completed all the steps in a content slide or not (Event 5). The problem slide events we consider are “Passed submission”/“Failed submission”/“No submission”, representing whether the student made a successful (Event 9), failed (Event 8) or no submission for the problem slide, which is not explicitly recorded as an interaction event by the platform.

The length of the feature vector is the total number of slides in the module, and the last entry is the outcome of the last problem slide, which typically makes use of the knowledge and skills from the previous slides in the module. The elements of the feature vector corresponding to content slides take values “Completed”/“Not completed”, and the elements corresponding to problem slides take values “Passed submission”/“Failed submission”/“No submission”.

We repeat this process for all modules and courses, creating separate datasets for each module. For a given module, we split its dataset of interaction vectors into separate training and test sets using 10-fold cross-validation with stratification. This ensures that data points from all students are used to train and validate the models.

From a machine learning point of view, the task is a classification problem. For a given student, we predict their performance on the last problem slide on a given module (“No submission”, “Failed submission” or “Passed submission”), given their interactions with the previous slides. Therefore, the target class is the performance on the last problem slide and the input features are the interaction vectors. We build a separate prediction model for each module.

The distribution of the three classes across all modules for each course are reported in Table 3. For most courses, except for Beginners Blockly, the percentage of students

who successfully complete the final problem slide tends to decrease as the course progresses.

## Feature Selection and Ranking

We use two feature selection algorithms: Correlation-Based Feature Selection (CFS) and Information Gain Ratio (GR) to select a subset of relevant slides for each module.

CFS (Hall & Holmes, 2003) selects a subset of features that are highly correlated with the target class but not correlated with each other, using a heuristic evaluation function which combines these two criteria to calculate the merit of each subset. To create the feature subsets, we used the best-first search option which starts with an empty set of features and adds new features.

GR (Witten et al., 2016) evaluates the value of a feature by measuring the gain ratio with respect to the target class and ranking the features based on it - the higher the value, the more informative the feature is. GR can also be used as an attribute selector for constructing decision trees - features with high GR tend to lead to purer splits in the decision tree, and therefore, smaller and more generalisable trees. For the GR feature selection method, we first looked at the ranking of the features produced by the algorithm and the feature scores, and decided to use the top 2–3 features based on the difference between the scores. Features with scores that were substantially lower than other features were excluded.

## Building Decision Trees

For each module, we trained a Decision Tree (DT) classifier under three conditions - without feature selection, with CFS and with GR feature selection.

**Table 3** Distribution of classes across modules for each course [%]

| Course              | Outcome | 1    | 2    | 3    | 4    | 5    | 6    | 7    | 8    | 9    | 10   |
|---------------------|---------|------|------|------|------|------|------|------|------|------|------|
| Newbies Python      | Passed  | 65.1 | 65.2 | 65.2 | 65.2 | 65.2 | 58.0 | 45.0 | 56.1 | 56.1 | 56.1 |
|                     | Failed  | 4.3  | 4.0  | 4.0  | 4.0  | 4.0  | 3.2  | 2.4  | 3.7  | 3.7  | 3.7  |
|                     | No sub  | 30.6 | 30.7 | 30.7 | 30.7 | 30.7 | 38.8 | 52.6 | 40.2 | 40.2 | 40.2 |
| Beginners Blockly   | Passed  | 44.2 | 50.6 | 53.7 | 53.7 | 53.7 | 53.7 | 53.7 | 53.3 | 53.3 | 53.3 |
|                     | Failed  | 2.0  | 2.5  | 2.5  | 2.5  | 2.5  | 2.5  | 2.5  | 2.4  | 2.4  | 2.4  |
|                     | No sub  | 53.7 | 46.8 | 43.7 | 43.7 | 43.7 | 43.7 | 43.7 | 44.2 | 44.2 | 44.2 |
| Beginners Python    | Passed  | 79.0 | 79.0 | 79.0 | 79.0 | 79.0 | 79.0 | 79.0 | 79.0 | 62.1 | 62.1 |
|                     | Failed  | 1.5  | 1.5  | 1.5  | 1.5  | 1.5  | 1.5  | 1.5  | 1.5  | 0.7  | 0.7  |
|                     | No sub  | 19.4 | 19.4 | 19.4 | 19.4 | 19.4 | 19.4 | 19.4 | 19.4 | 37.2 | 37.2 |
| Intermediate Python | Passed  | 77.9 | 77.9 | 77.9 | 77.9 | 76.3 | 74.6 | 74.6 | 74.6 | 74.6 | 74.6 |
|                     | Failed  | 0.8  | 0.8  | 0.8  | 0.8  | 1.2  | 1.2  | 1.2  | 1.2  | 1.2  | 1.2  |
|                     | No sub  | 21.2 | 21.2 | 21.2 | 21.2 | 22.4 | 24.0 | 24.0 | 24.0 | 24.0 | 24.0 |

**Table 4** Time gap between last visit of preceding slide and deadline for the last problem [hours]

| Course              | Mean  | Std  | Median | IQR   | Q1    | Q3    |
|---------------------|-------|------|--------|-------|-------|-------|
| Newbies Blockly     | 171.8 | 86.0 | 179.6  | 145.6 | 102.1 | 247.7 |
| Beginners Blockly   | 166.7 | 77.3 | 176.2  | 115.0 | 108.4 | 223.3 |
| Beginners Python    | 177.7 | 73.3 | 184.1  | 107.8 | 126.6 | 234.5 |
| Intermediate Python | 201.3 | 88.4 | 196.1  | 128.3 | 142.8 | 271.1 |

DTs are intrinsically interpretable classifiers by design. They can be represented as a set of rules and provide an easy to understand tree-based visualisation. They can also be seen as conducting a secondary feature selection - only the most important features will appear in the tree and the features closer to the root are more important than the ones closer to the leaves. Each rule can also be assessed based on its coverage and accuracy. This intrinsic interpretability is in contrast with recently proposed post-hoc explainer methods such as SHAP (Lundberg & Lee, 2017) and LIME (Ribeiro et al., 2016) which analyse the predictions of black-box classifiers and assign importance values to the features to help understand what has been learned.

DTs are highly appropriate for our context as the generated set of rules provide an explanation to teachers for why the students were assigned to a particular class (Passed/Failed/No Submission) and identify the important features affecting students' performance. They are also computationally efficient classifiers - fast to train and fast to make predictions.

We compared the DT results with a baseline which predicts the majority class, and three other machine learning algorithms: Logistic Regression (LR), Random Forest (RF) and Support Vector Machine (SVM). While LR can be considered partially explainable since the regression coefficients can be interpreted as feature weights, SVM and RF are intrinsically black-box models. Post-hoc explainers such as SHAP and LIME can be used to provide insights on how the decisions are made but the generated explanations are limited to feature importance. These explainers also require additional time to run and are less suitable for non-technical users. As reported by Retzlaff et al. Retzlaff et al. (2024), both technical and non-technical users found the default presentation of SHAP results difficult to understand and utilize. In contrast, DT provides a set of rules which are easy to understand and are also more informative, compared to feature importance ranking only.

**Table 5** Percentage of students who reach the second last slide and the last slide for each course

| Challenge           | Ran 2nd Last Slide [%] | Ran Last Slide [%] |
|---------------------|------------------------|--------------------|
| Newbies Blockly     | 83.7                   | 91.9               |
| Beginners Blockly   | 78.4                   | 90.0               |
| Beginners Python    | 88.3                   | 94.6               |
| Intermediate Python | 95.6                   | 91.9               |

## Prediction Window and Intervention Opportunity

We investigate the time available for intervention. Table 4 shows the time gap between a student's last visit of the preceding slide, after which the prediction is made, and the deadline for the final slide, which is the slide being predicted. We can observe that the proposed method has a sufficiently high prediction window, with the median ranging between 176.2 hours (7.3 days) and 196.1 hours (8.2 days). Table 5 shows that most students reach the second last slide, and most of these students then proceed to attempt the last problem slide. The large prediction window gives teachers an opportunity to intervene in real-time, for example by identifying struggling students and encouraging them to go back and work through an important slide that is key to affecting the final outcome.

Since DTs are essentially instantaneous to evaluate, predictions can be made in real time, and could be used during a school classroom session or displayed as feedback when the student is using the system. The DT models need to be constructed based on prior data. However, many course modules are re-used over the years with only surface-level changes each year, and so the associated DTs would be the same. For newer content, DT models could be constructed from the outcomes of early users and made available for use by subsequent users. DTs would be suitable for this scenario as they are fast to train, in comparison to other models such as neural networks, RF and SVM.

## Implementation

All data preparation and processing was done in Python. The feature selection and classification were done using the Weka's implementation of the algorithms (Witten et al., 2016). We used Weka's default hyperparameters and settings for all algorithms. For Logistic Regression (LR), this included the default number of updates set to  $-1$ , which allows the algorithm to update until convergence. For Random Forest (RF), we used a default batch size of 100 for batch prediction and a minimum of one instance per leaf. For Support Vector Machines (SVM), we employed Weka's implementation of the Sequential Minimal Optimisation (SMO) algorithm, with the default complexity  $C = 1$ , a tolerance parameter  $L = 1.0 \times 10^{-3}$ , and normalisation enabled.

## Results

Table 6 shows the predictive performance results for each course, averaged over the 10 modules comprising the course. We report the overall accuracy across all classes, as well as the precision, recall and F1 scores.

In addition, for the DT approaches, we report two other metrics. "Leaves" is the number of leaf nodes in tree; each path from the root to a leaf node corresponds to one classification rule extracted from the tree. "Size" is the number of all nodes in the tree - non-leaf (attribute-value tests) and leaf (class values). The number of leaves and tree size are indicators of model complexity as well as interpretability. In general,

**Table 6** Predictive performance (accuracy, precision, recall and F1 score) and DT characteristics

|                            | Predictive performance metrics |           |        |      | DT metrics |      |
|----------------------------|--------------------------------|-----------|--------|------|------------|------|
|                            | Accuracy                       | Precision | Recall | F1   | Leaves     | Size |
| <b>Novice Blockly</b>      |                                |           |        |      |            |      |
| Baseline (Majority class)  | 58.5                           | 0.58      | 0.22   | 0.73 |            |      |
| LR                         | 83.8                           | 0.81      | 0.84   | 0.81 |            |      |
| RF                         | 83.3                           | 0.81      | 0.83   | 0.80 |            |      |
| SVM                        | 83.9                           | 0.91      | 0.84   | 0.87 |            |      |
| DT                         | 85.3                           | 0.90      | 0.78   | 0.83 | 6.1        | 9.9  |
| DT + CFS                   | 85.2                           | 0.89      | 0.79   | 0.83 | 4.3        | 6.3  |
| DT + GR                    | 85.3                           | 0.89      | 0.79   | 0.83 | 4.1        | 5.9  |
| <b>Beginners Blockly</b>   |                                |           |        |      |            |      |
| Baseline                   | 60.4                           | 0.61      | 0.80   | 0.76 |            |      |
| LR                         | 93.6                           | 0.93      | 0.94   | 0.93 |            |      |
| RF                         | 94.5                           | 0.93      | 0.95   | 0.94 |            |      |
| SVM                        | 93.7                           | 0.93      | 0.94   | 0.93 |            |      |
| DT                         | 88.8                           | 0.92      | 0.92   | 0.92 | 7.6        | 11.9 |
| DT + CFS                   | 88.9                           | 0.89      | 0.93   | 0.91 | 4.2        | 5.9  |
| DT + GR                    | 86.9                           | 0.90      | 0.92   | 0.90 | 3.9        | 5.4  |
| <b>Beginners Python</b>    |                                |           |        |      |            |      |
| Baseline                   | 71.7                           | 0.70      | 0.90   | 0.81 |            |      |
| LR                         | 94.5                           | 0.85      | 0.95   | 0.94 |            |      |
| RF                         | 94.1                           | 0.93      | 0.94   | 0.94 |            |      |
| SVM                        | 94.4                           | 0.94      | 0.94   | 0.94 |            |      |
| DT                         | 90.8                           | 0.89      | 0.95   | 0.91 | 13.9       | 23.6 |
| DT + CFS                   | 90.8                           | 0.89      | 0.95   | 0.91 | 4.7        | 6.8  |
| DT + GR                    | 90.9                           | 0.88      | 0.98   | 0.92 | 5.0        | 7.0  |
| <b>Intermediate Python</b> |                                |           |        |      |            |      |
| Baseline                   | 71.8                           | 0.72      | 0.70   | 0.83 |            |      |
| LR                         | 90.3                           | 0.89      | 0.90   | 0.89 |            |      |
| RF                         | 89.3                           | 0.88      | 0.89   | 0.88 |            |      |
| SVM                        | 90.3                           | 0.84      | 0.90   | 0.87 |            |      |
| DT                         | 88.9                           | 0.89      | 0.91   | 0.89 | 7.8        | 13.3 |
| DT + CFS                   | 88.9                           | 0.89      | 0.92   | 0.90 | 3.4        | 4.7  |
| DT + GR                    | 88.4                           | 0.88      | 0.92   | 0.90 | 3.2        | 4.3  |

small trees with few leaves are less complex and more interpretable. In particular, compact trees that also exhibit strong accuracy performance indicate that the model has performed well – it is able to learn the important decision rules from the data and can generalise well on unseen data.

The DTs with and without feature selection show consistently high predictive performance on all courses - 85.2-90.7% accuracy, precision of 0.88-0.92 precision, 0.78-0.98 recall and 0.83-0.92 F1 score. All DTs considerably surpass the majority class baseline. The machine learning methods used for comparison - LR, RF and SVM, performed slightly better than DTs, e.g. the best performing method SVM achieved 83.9-94.4 accuracy, 0.84-0.94 precision, 0.84-0.94 recall and 0.87-0.94 F1 score. These methods however lack interpretability, and SVM and LR were also slow to train which limits their applicability for our case study.

Table 7 shows the predictive performance of the DT models for each class. The DTs achieved high performance when predicting “Passed” and “No submission” scoring, scoring high on precision, recall and F1 across all courses. However, predicting class “Failed” remains challenging, with significantly lower precision, recall and F1 scores

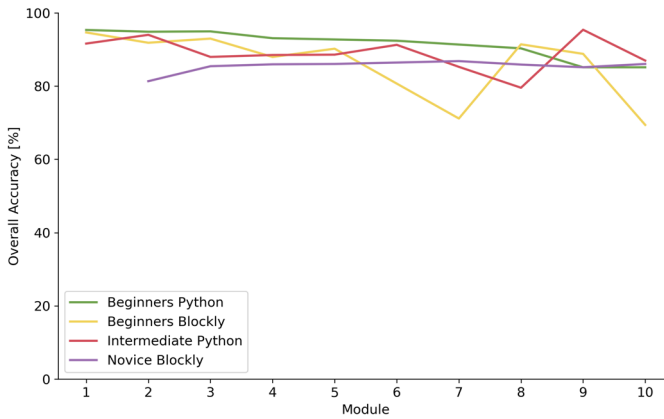
**Table 7** Predictive performance for each class (precision, recall and F1 score) for the DT models

|                     | Predictive performance metrics |        |      |
|---------------------|--------------------------------|--------|------|
|                     | Precision                      | Recall | F1   |
| DT                  |                                |        |      |
| Novice Blockly      |                                |        |      |
| Passed              | 0.90                           | 0.78   | 0.83 |
| Failed              | 0.28                           | 0.08   | 0.21 |
| No submission       | 0.77                           | 0.86   | 0.80 |
| Beginners Blockly   |                                |        |      |
| Passed              | 0.92                           | 0.92   | 0.92 |
| Failed              | 0.24                           | 0.03   | 0.08 |
| No submission       | 0.86                           | 0.91   | 0.88 |
| Beginners Python    |                                |        |      |
| Passed              | 0.89                           | 0.95   | 0.91 |
| Failed              | 0.38                           | 0.04   | 0.08 |
| No Submission       | 0.92                           | 0.80   | 0.85 |
| Intermediate Python |                                |        |      |
| Passed              | 0.89                           | 0.91   | 0.89 |
| Failed              | 0.12                           | 0.01   | 0.01 |
| No Submission       | 0.89                           | 0.79   | 0.83 |
| DT+GR               |                                |        |      |
| Novice Blockly      |                                |        |      |
| Passed              | 0.89                           | 0.79   | 0.83 |
| Failed              | 0.38                           | 0.07   | 0.28 |
| No submission       | 0.83                           | 0.86   | 0.89 |

**Table 7** continued

|                     | Predictive performance metrics |        |      |
|---------------------|--------------------------------|--------|------|
|                     | Precision                      | Recall | F1   |
| Beginners Blockly   |                                |        |      |
| Passed              | 0.90                           | 0.92   | 0.90 |
| Failed              | 0.03                           | 0.00   | 0.01 |
| No submission       | 0.92                           | 0.85   | 0.82 |
| Beginners Python    |                                |        |      |
| Passed              | 0.88                           | 0.98   | 0.92 |
| Failed              | 0.61                           | 0.05   | 0.13 |
| No Submission       | 0.91                           | 0.78   | 0.83 |
| Intermediate Python |                                |        |      |
| Passed              | 0.88                           | 0.92   | 0.90 |
| Failed              | 0.00                           | 0.00   | 0.00 |
| No Submission       | 0.88                           | 0.75   | 0.80 |
| DT+CFS              |                                |        |      |
| Novice Blockly      |                                |        |      |
| Passed              | 0.89                           | 0.79   | 0.83 |
| Failed              | 0.95                           | 0.08   | 0.81 |
| No submission       | 0.83                           | 0.84   | 0.88 |
| Beginners Blockly   |                                |        |      |
| Passed              | 0.89                           | 0.93   | 0.91 |
| Failed              | 0.22                           | 0.02   | 0.09 |
| No submission       | 0.84                           | 0.85   | 0.83 |
| Beginners Python    |                                |        |      |
| Passed              | 0.89                           | 0.95   | 0.91 |
| Failed              | 0.49                           | 0.03   | 0.09 |
| No Submission       | 0.91                           | 0.81   | 0.85 |
| Intermediate Python |                                |        |      |
| Passed              | 0.88                           | 0.92   | 0.90 |
| Failed              | 0.11                           | 0.00   | 0.01 |
| No Submission       | 0.89                           | 0.78   | 0.82 |

across all courses. This may be attributed to the low representation of failed submissions in the training dataset (Table 3), with failed submissions accounting for at most 4.3% of submissions for any given module. For example, the Beginners Python course has the lowest percentage of examples from class “Failed” and the lowest predictive performance for this class. With fewer examples of failed submissions, the model may not have seen enough variations of slide features associated with failed submissions, leading to poor generalisation. The limited representation of failed submissions is due

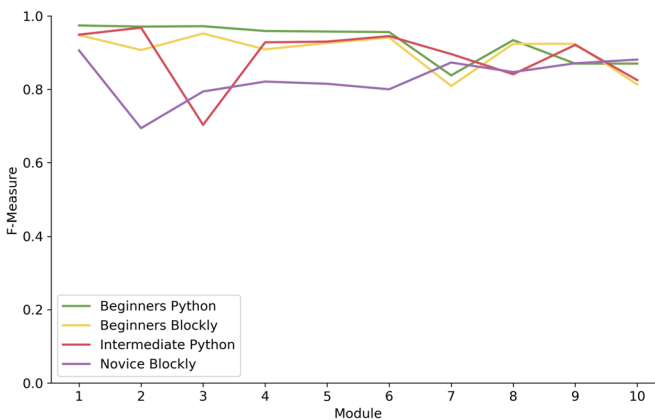


**Fig. 7** Accuracy for DT+GR classifier

to the coding problems being designed so that most students who attempt them can solve them.

The DT classifiers, with and without feature selection, performed similarly in terms of all predictive measures. This shows that the application of IG and CFS feature selection methods was beneficial - they were able to reduce the number of features while maintaining high predictive performance.

We now examine the performance across the modules using the DT classifier with GR feature selection as an example. Fig. 7 shows the accuracy for each module and Fig. 8 the F1 scores. Across all courses, accuracy remains consistently high (80% and above) for most modules up to Module 6, after which it tends to decrease (Fig. 7). Similarly, the F1 scores are generally high across modules and courses, with a slight drop towards the end of each course. These drops in accuracy and F1 towards the end may be attributed to student dropout in later modules across all courses (Table 3).



**Fig. 8** Average F1 score for DT+GR classifier for all classes

## Discussion

### Performance

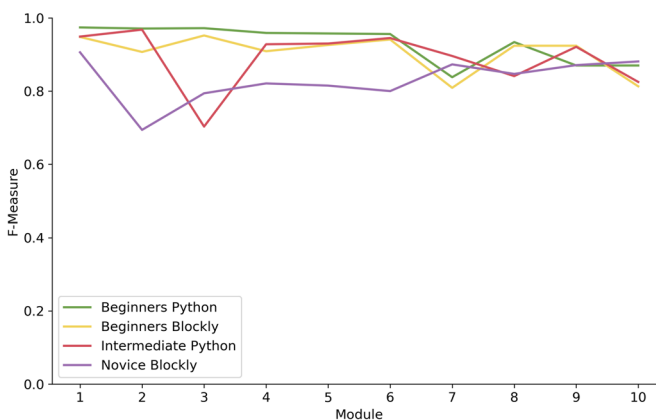
in predicting end of module outcomes

#### Q1: Are Slide Interaction Features a Useful Predictor of End of Module Outcomes?

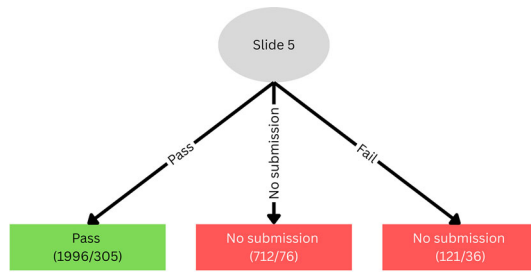
Our proposed DT classifier, which has been trained on slide interaction features, achieves high performance across all courses, with competitive accuracy, precision, recall and F1 scores compared to other models (Table 6). The performance of all DTs is consistently high across all four courses with or without feature selection, with both CFS and GR performing similarly. These results show that slide interaction features are both accurate and valuable for predicting student outcomes. In particular, the strong performance in identifying cases of “No submission”, as seen in Table 7 and Fig. 9, suggests this method can be used to flag students who may be struggling with the module and at risk of dropping out before course completion. Importantly, educators have a window of 7 to 8 days between the last visit on the previous slide and the deadline for the final problem, which provides an opportunity for real-time intervention (Table 4).

### Effects of Feature Selection

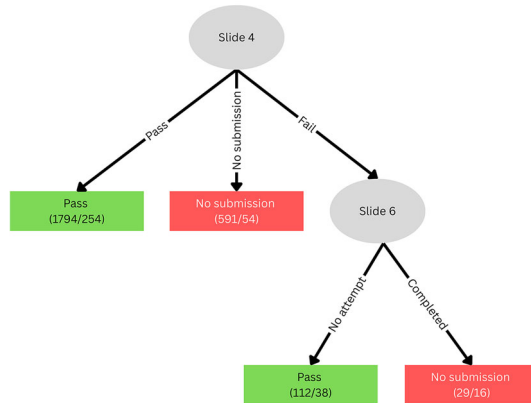
**Q2: How can we Represent Student Progress in a Useful Format for Educators?** The two feature selection methods (CFS and GR) created decision trees with fewer leaf nodes while maintaining accuracy. Compared to the decision trees without feature selection, which had on average 6.1–13.9 leaf nodes, using the CFS and GR feature selection methods reduced the number of leaf nodes to 3.4–4.7 and 3.2–4.1 without degrading performance (Table 6). Both CFS and GT produced trees of similar size,



**Fig. 9** F1 score for DT+GR classifier for class “No submission”



(a) Intermediate Python w4p1 (with GR feature selection)



(b) Novice Blockly w2p1 (with GR feature selection)

**Fig. 10** Example decision trees

with GT generating slightly smaller trees - average tree size of 5.7 and 4 leaf nodes for GT, compared to 5.9 and 4.1 for CFS. This demonstrates that, through feature selection, we can create a much smaller and more interpretable tree with less paths (as indicated by the reduced number of leaf nodes) without sacrificing performance, and in some cases, even improving it. This result may be useful for educators - they can use the decision trees to gain a better understanding of how their students progress throughout the course, and identify key points where intervention may be beneficial. Additionally, this result points to the need to have slide-specific features rather than aggregate models, since often a single slide is pivotal.

For instance, the decision tree in Fig. 10b shows two paths leading to a “Pass” outcome. To obtain “Pass” students should complete Slide 4. To avoid “No submission” on the last problem slide, a student that has attempted but not passed Slide 4 should consider completing Slide 6 to obtain “Pass”. This analysis reveals two key points in the course, Slide 4 and 6, where teachers may wish to intervene. A smaller tree makes it easier for educators to identify key areas where students may need intervention. It also makes understanding student learning journeys clearer for educators.

## Key Observations on Slides Chosen by the Feature Selection Algorithm

### Q3: What are the Important Slides in Each Module?

**Observation 1:** Completion of prior exercises in a module is the most critical predictor of completing the module's final exercise. On average, problem slides make up 75% of features selected by the GR feature selection algorithm across modules. Students who complete problem slides, particularly later ones, are more likely to pass the last problem slide in the module, and students who do not make a submission or fail the problem slide are likely to make no submission on the final problem slide. This is because earlier problem slides tend to mirror later problem slides, and contain important information and steps that are required to complete remaining problem slides. These results are consistent with the “doer effect” (Koedinger et al., 2016) which posits that interactive practice is more beneficial than passive reading and video watching. These results also suggest that students may benefit from focusing their efforts on the problem slides within each module, particularly later problem slides.

**Observation 2:** Some interactive content can also be strongly influential on module outcomes. Although less represented among slide features chosen by the feature selection algorithm, interactive content slides are also selected for several modules, particularly in the Novice Blockly and Intermediate Python courses, for which interactive content slides account for 25% and 40% of chosen slide features on average across all modules for these courses respectively. This suggests content slides may play a useful role in providing programming knowledge for students.

Content slides provide useful information that sets the student up for success in the module, such as introducing new concepts e.g. nested functions, Python dictionaries and immutability. Content slides play a more important role in the Intermediate Python course, which is the most advanced out of the four courses, which is reasonable since it contains more advanced slide content that students would be more inclined to engage with.

In addition, content slides with runnable code seem to yield better outcomes and students who view but not run the code tend to have poorer outcomes. This highlights the importance of active interaction with code examples, and is a motivator for methodologies such as PRIMM (Sentance et al., 2019). Lastly, some newly introduced features on the platform, such as interactive steps and paired problems seem to have value, as these appear to show up as higher-ranked slides chosen by the feature selection algorithm.

**Observation 3:** The more advanced students in the Beginners Python course find greater value in completing problem slides than content slides, while the less advanced students in the Beginners Blockly course find greater value in completing content slides. As seen in Tables 8 and 9, there are several differences between the slides selected by the GR feature selection algorithm for the Beginners Blockly course and its corresponding Python version, the Beginners Python course. The feature selection algorithm selects mostly problem slides (23/27 slides) for Beginners Python and

**Table 8** Comparison of slides selected by the feature selection algorithm for Beginners Blockly and Beginners Python (first half of the course)

| Module | Slide | Beginners Blockly                    | Beginners Python                     | Slide type |
|--------|-------|--------------------------------------|--------------------------------------|------------|
| w1p1   | 3     | What happens when things go wrong    | —                                    | Content    |
|        | 7     | Using multiple print statements      | —                                    | Content    |
|        | 9     | <b>Breaking news</b>                 | <b>Breaking news</b>                 | Problem    |
|        | 13    | Calling functions                    | —                                    | Content    |
|        | 16    | —                                    | Stage names                          | Problem    |
| w1p2   | 1     | <b>Soup of the day</b>               | <b>Soup of the day</b>               | Content    |
|        | 7     | <b>Collecting pikachu</b>            | <b>Collecting pikachu</b>            | Problem    |
|        | 12    | User input with the turtle           | —                                    | Content    |
|        | 14    | —                                    | Can turtles climb                    | Problem    |
| w2p1   | 3     | Assignment vs comparison             | —                                    | Content    |
|        | 4     | —                                    | Shotgun                              | Problem    |
|        | 7     | If it isn't raining...               | —                                    | Content    |
|        | 8     | —                                    | I Am Katniss                         | Problem    |
|        | 11    | Making decisions with numbers        | —                                    | Content    |
|        | 12    | —                                    | Basketball Bunny!                    | Problem    |
| w2p2   | 2     | More on quotes                       | —                                    | Content    |
|        | 3     | Multiplying strings                  | —                                    | Content    |
|        | 4     | —                                    | How many yips?                       | Problem    |
|        | 6     | Working with numbers and strings     | —                                    | Content    |
|        | 7     | —                                    | When Life Gives You Lemons           | Problem    |
|        | 11    | —                                    | Left, or Right?                      | Problem    |
| w3p1   | 3     | <b>Frogkisser</b>                    | <b>Frogkisser</b>                    | Problem    |
|        | 9     | <b>Title Tangle</b>                  | <b>Title Tangle</b>                  | Problem    |
|        | 12    | <b>Dot the I's and cross the t's</b> | <b>Dot the I's and cross the t's</b> | Problem    |
| w3p2   |       | <b>Spell it out for me</b>           | <b>Spell it out for me</b>           | Problem    |
|        |       | <b>Lift off!</b>                     | <b>Lift off!</b>                     | Problem    |
|        |       | Pen up and down                      | —                                    | Content    |
|        |       | —                                    | Stairway to Heaven                   | Problem    |

content slides (18/29) for Beginners Blockly. For the more advanced students in the Beginners Python course, this could reflect skipping slides after viewing and partially attempting earlier ones, possibly because they feel the course is too basic for their skill level (Jeffries et al., 2020). For the less advanced students in the Beginners Blockly course, this bigger reliance on content slides could reflect less background knowledge and a greater need to master fundamental content.

**Table 9** Comparison of slides selected by the feature selection algorithm for Beginners Blockly and Beginners Python (second half of the course)

| Module | Slide | Beginners Python               | Beginners Blockly          | Slide type |
|--------|-------|--------------------------------|----------------------------|------------|
| w3p2   | 2     | <b>Spell it out for me</b>     | <b>Spell it out for me</b> | Problem    |
|        | 7     | <b>Lift off!</b>               | <b>Lift off!</b>           | Problem    |
|        | 11    | Pen up and down                | —                          | Content    |
|        | 12    | —                              | Stairway to Heaven         | Problem    |
| w4p1   | 2     | —                              | Secret Yaks                | Problem    |
|        | 3     | Negative Indexes               | —                          | Content    |
|        | 4     | —                              | Extra emphasissssss!       | Problem    |
|        | 5     | Representing information       | —                          | Content    |
|        | 8     | —                              | Representation in binary   | Problem    |
|        | 13    | Secret Recipe                  | —                          | Problem    |
| w4p2   | 3     | Strings cannot be changed      | —                          | Content    |
|        | 4     | —                              | The Problem with Prefixes  | Problem    |
|        | 6     | Making decisions while looping | —                          | Content    |
|        | 8     | —                              | Run, Atlas, Run!           | Problem    |
|        | 10    | Choosing the fill colour       | —                          | Content    |
|        | 12    | —                              | Green Diamonds             | Problem    |
| w5p1   | 7     | <b>Making Headlines!</b>       | <b>Making Headlines!</b>   | Content    |
|        | 12    | <b>Function Vowels</b>         | <b>Function Vowels</b>     | Problem    |
| w5p2   | 4     | <b>Rectangul-Art!</b>          | <b>Rectangul-Art!</b>      | Problem    |
|        | 5     | <b>Swatch This!</b>            | <b>Swatch This!</b>        | Problem    |

## Evaluation by Educators

**Q4: To What Extent is this Approach and the Results Useful for Educators?** The proposed approach and results were evaluated by two educators who were experienced educators who had developed content for the courses used in the study. One was a university lecturer with experience teaching computer science to tertiary education students, while the other was formerly a high school teacher with a specialisation in digital technologies education for school years 7–10. In total, the educators evaluated the results for 14 separate modules from the Novice Blockly, Beginners Blockly and Beginners Python courses, and found the trees useful.

The first educator found that “the tree does provide a useful means to identify the key places to flag that a student may be struggling and likely to give up”. The educator also noted that the tree could potentially be used to “identify slides that are redundant or have little pedagogical value, so that they could be removed or improved”. In addition, slides that are particularly predictive of failure could be used to “flag to students that they should not skip specific steps”, and to “notify educators that they should consider intervening for some students at risk of future failure”. One drawback that the educator

noted was that the “nested trees could be misleading as educators should not think that slides that were not ranked should be removed”.

The second educator found that the results matched their understanding of the students and courses, noting that they also expected that “a majority of students who finish an problem slide earlier in a module (would) also be able to complete it later”. In particular, they found it “incredibly useful to know which content slide (or problem slide) is a general indicator that the student will complete the module.” One improvement that they suggested was for the algorithm to show slides with the lowest slide scores (i.e. those that students are likely to skip or find difficult to understand), so that educators can improve the slides for future iterations of the course.

## Conclusion

In this paper, we proposed a DT based approach to provide accurate and interpretable predictions of student progress at a module level. Our approach predicts the student performance on the last problem in the module, based on the student interactions with the previous content in the module. We define and extract suitable content interaction features from the student log data, and then select the most important feature subset using the CFS and GR algorithms to create DTs. The evaluation was conducted on four large scale programming course for upper primary and high school students at different levels. We found that this approach can lead to accurate predictions of how students would perform at the end of each module, as well as easy to understand tree-based visualisations that, along with feature rankings, reveal the key slides that students should attempt to pass the module successfully. In addition, our analysis found key observations about course slides, in particular, that problem slides were by far the most important predictor of end of module success. It also identifies key places where students might be struggling and give up, and programming tasks that are predictive of failure and success and should be completed. We also found that there were distinct differences between the Beginners Blockly and Beginners Python courses, with the Python course students relying less on content slides.

We chose DTs as they are intrinsically interpretable classifiers, fast to train and make predictions. They were able to provide high predictive performance, competitive with black-box models such as RF, SVM, and also LR. In the last few years, there has been lot of interest in using complex black-box models with post-hoc explainers. The choice of method ultimately depends on the specific application and required balance between accuracy and interpretability, but complex models may not be needed and there should be a comparison with glass-box models such as DTs.

Our approach was motivated in part by our initial observations of student interaction patterns, for which we found three distinct groups of students - 1) at-risk, disengaged students for which early detection is critical – this motivated our reporting of “No submission” accuracy along with overall accuracy, 2) disengaged but high performing students that provided useful context in showing that a lack of content slide engagement could be linked to both negative and positive student programming outcomes, and 3) engaged and high-performing students who showed that among some students,

completing content slides can be tied to strong performance on final problem slides, and that engagement with content slides can be one effective predictor of end of module outcomes.

Overall, we think there is an opportunity for educators who are using or interested in online programming courses to adopt our approach to better understand student programming progress. As noted by the educators who reviewed our approach, educators could use the DTs to better visualise how students are progressing through the course, and identify exercises and topics that students might be missing or struggling with. Educators could also use the slide rankings to identify slides that offer little pedagogical value so they can be improved or removed. They can also use our approach to identify differences in student behaviour between courses - e.g. between different course levels (beginners vs intermediate) or versions (blockly vs non-blockly).

However, we note that predicting failed submissions remains a challenge. One limitation of our data is the lack of sufficient failed submissions for later courses. In this case, the models may not have seen sufficient examples of failed submissions and their corresponding slide interaction features to be able to generalise well on unseen data.

In future work, we intend to investigate the value of the slides prior to key slides that were included in the decision trees, and those that were not ranked highly (e.g. slides that did not make it into the highest ranked slides). This would allow us to better understand progression at each step of the module and show the influence of completing (or not completing) each step. In addition, we intend to classify the problem slide types further, as there are several different problem slide types such as Parsons problems and paired problems where the solution of one problem is strongly connected to that of the next. Knowing the value and importance of different types of problem slides could encourage further innovation in problem slides, and allow educators and course content creators to more effectively design problem slides for students. Also, we hope to include more data around the order in which students attempted the slides. This may provide a more nuanced understanding of how the sequencing of slide and problem slide attempts affects a student's likelihood of success. Another avenue for future work is investigating the use of the first  $n$  slides to predict the outcome of the last slide, progressively increasing  $n$ , to determine how early an accurate prediction can be made.

**Author Contributions** Study conception and design: V.Z., B.J., I.K.; Data preparation: V.Z., B.J.; Software development: V.Z.; Data analysis and interpretation: V.Z., B.J., I.K.; Manuscript preparation: V.Z., B.J., I.K. All authors approved the final manuscript.

**Funding** Open Access funding enabled and organized by CAUL and its Member Institutions.

**Data Availability** Due to the proprietary nature of the data, the dataset used in this research is not available publicly.

## Declarations

**Competing interests** The authors declare no competing interests.

**Open Access** This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

## References

- Abdelrahman, G., Wang, Q., Nunes, B. (2023). Knowledge tracing: A survey. *ACM Computing Surveys*, 55(11)
- Berens, J., Schneider, K., Gortz, S., Oster, S., & Burghoff, J. (2019). Early detection of students at risk - predicting student dropouts using administrative student data from german universities and machine learning methods. *Journal of Educational Data Mining*, 11(3), 1–41.
- Bonar, J., & Soloway, E. (1985). Preprogramming knowledge: A major source of misconceptions in novice programmers. *Human-Computer Interaction*, 1(2), 133–161.
- Conijn, R., Snijders, C., Kleingeld, A., & Matzat, U. (2017). Predicting student performance from lms data: A comparison of 17 blended courses using moodle lms. *IEEE Transactions on Learning Technologies*, 10(1), 17–29.
- Corbett, A. T., & Anderson, J. R. (1995). Knowledge tracing: Modelling the acquisition of procedural knowledge. *User Modelling and User-Adapted Interaction*, 4(4), 253–278.
- Dale, N. B. (2006). Most difficult topics in CS1: Results of an online survey of educators. *ACM SIGCSE Bulletin*, 38(2), 49–53.
- Dsilva, V., Schleiss, J., Stober, S. (2023). Trustworthy academic risk prediction with explainable boosting machines. In: *Proceedings of the International Conference on Artificial Intelligence in Education*, pp. 463–475
- Hall, M. A., & Holmes, G. (2003). Benchmarking attribute selection techniques for discrete class data mining. *IEEE Transactions on Knowledge and Data Engineering*, 15(6), 1437–1447.
- Jeffries, B., Baldwin, T., Zalk, M., Taylor, B. (2020). Online tutoring to support programming exercises. In: *Proceedings of the Twenty-Second Australasian Computing Education Conference*, pp. 56–65
- Kaser, T., Klingler, S., Schwing, A. G., & Gross, M. (2017). Dynamic Bayesian networks for student modeling. *IEEE Transactions on Learning Technologies*, 10(4), 450–462.
- Kemper, L., Vorhoff, G., & Wigger, B. U. (2020). Predicting student dropout: A machine learning approach. *European Journal of Higher Education*, 10(1), 28–47.
- Koedinger, K. R., McLaughlin, E. A., Jia, J. Z., Bier, N. L. (2016). Is the doer effect a causal relationship? In: *Proceedings of the International Conference on Learning Analytics and Knowledge*, pp. 388–397
- Koprinska, I., Stretton, J., Yacef, K. (2015). Predicting student performance from multiple data sources. In: *Proceedings of the International Conference on Artificial Intelligence in Education*, pp. 678–681
- Koprinska, I., Stretton, J., Yacef, K. (2015). Students at risk: detection and remediation. In: *Proceedings of the International Conference on Educational Data Mining*, pp. 512–515
- Kuzilek, J., Hlosta, M., Herrmannova, D., Zdrahal, Z., Vaclavek, J., Wolff, A. (2015). Ou analyse: analysing at-risk students at the open university authors. *Learning Analytics Review*, pp. 1–16
- Kuzilek, J., Hlosta, M., Zdrahal, Z. (2017) Open university learning analytics dataset. *Scientific Data*. (4)
- Lundberg, S. M., Lee, S.-I. (2017). A unified approach to interpreting model predictions. In: *Proceedings of the International Conference on Neural Information Processing Systems*, pp. 4765–4774
- Lykourantzou, I., Giannoukos, I., Nikolopoulos, V., Mpardis, G., & Loumos, V. (2009). Dropout prediction in e-learning courses through the combination of machine learning techniques. *Computers & Education*, 53(3), 950–965.
- McBroom, J., Jeffries, B., Koprinska, I., Yacef, K. (2016). Mining behaviors of students in autograding submission system logs. In: *Educational Data Mining*, pp. 159–166
- McBroom, J., Paassen, B., Jeffries, B., Koprinska, I., Yacef, K. (2021). Progress networks as a tool for analysing student programming difficulties. In: *Proceedings of the Australasian Conference on Computing Education*, pp. 158–167

- McBroom, J., Yacef, K., Koprinska, I. (2020). Detect: A hierarchical clustering algorithm for behavioural trends in temporal educational data. In: *Artificial Intelligence in Education*, pp. 374–385. Springer, Cham
- Pardos, Z. A., Heffernan, N. T. (2010). Modeling individualization in a Bayesian networks implementation of knowledge tracing. In: De Bra, P., Kobsa, A., Chin, D. (eds.) *Proceedings of the 18th International Conference on User Modeling, Adaptation, and Personalization (UMAP)*, pp. 255–266
- Pereira, F. D., Oliveira, E., Cristea, A., Fernandes, D., Silva, L., Aguiar, G., Alamri, A., Alshehri, M. (2019). Early dropout prediction for programming courses supported by online judges. In: *Proceedings of the International Conference on Artificial Intelligence in Education*, pp. 67–72
- Perera, D., Kay, J., Koprinska, I., Yacef, K., & Zaïane, O. (2009). Clustering and sequential pattern mining of online collaborative learning data. *IEEE Transactions on Knowledge and Data Engineering.*, 21(6), 759–772.
- Piech, C., Bassen, J., Huang, J., Ganguli, S., Sahami, M., Guibas, L., Sohl-Dickstein, J. (2015). Deep knowledge tracing. In: *Proceedings of the International Conference on Neural Information Processing Systems*, pp. 505–513
- Retzlaff, C. O., Angerschmid, A., Saranti, A., Schneeberger, D., Röttger, R., Müller, H., Holzinger, A. (2024). Post-hoc vs ante-hoc explanations: xAI design guidelines for data scientists. *Cognitive Systems Research*, 86
- Ribeiro, M. T., Singh, S., Guestrin, C. (2016). Why should I trust you?: Explaining the predictions of any classifier. In: *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 1135–1144
- Riestra-González, M., Puerto Paule-Ruiz, M., & Ortin, F. (2021). Massive lms log data analysis for the early prediction of course-agnostic student performance. *Computers & Education.*, 163, Article 104108.
- Romero, C., López, M.-I., Luna, J.-M., & Ventura, S. (2013). Predicting students' final performance from participation in on-line discussion forums. *Computers & Education.*, 68, 458–472.
- Sentance, S., Waite, J., Kallia, M. (2019). Teachers' experiences of using primm to teach programming in school. In: *Proceedings of the 50th ACM Technical Symposium on Computer Science Education*, pp. 476–482. ACM, New York, NY, USA
- Tomasevic, N., Gvozdenovic, N., & Vranes, S. (2020). An overview and comparison of supervised data mining techniques for student exam performance prediction. *Computers & Education.*, 143, Article 103676.
- Vegas, E., Hansen, M., & Fowler, B. (2021). *How to expand and improve computer science education around the world*. Brookings Institution: Technical report.
- Wagner, K., Volkening, H., Basyigit, S., Merceron, A., Sauer, P., Pinkwart, N. (2023). Which approach best predicts dropouts in higher education? In: *International Conference on Computer Supported Education*, pp. 15–26
- Weintrop, D., Wilensky, U. (2017). Comparing block-based and text-based programming in high school computer science classrooms. *ACM Transactions on Computing Education*, 18(1)
- Winslow, L. E. (1996). Programming pedagogy—a psychological overview. *ACM SIGCSE Bulletin.*, 28, 17–22.
- Witten, I. H., Frank, E., Hall, M. A., & Pal, C. J. (2016). *Data Mining: Practical Machine Learning Tools and Techniques* (4th ed.). San Francisco, CA, USA: Morgan Kaufmann Publishers Inc.
- Yudelson, M. V., Koedinger, K. R., Gordon, G. J. (2013). Individualized Bayesian knowledge tracing models. In: *Proceedings of the International Conference on Artificial Intelligence in Education*, pp. 171–180
- Zhang, V., Jeffries, B., Koprinska, I. (2023). Predicting progress in a large-scale online programming course. In: *Proceedings of the International Conference on Artificial Intelligence in Education*, pp. 810–816